

Teaching Philosophy

My central goal as an educator is to help students develop **intellectual independence**. This goal is closely connected to my own learning journey. Earlier in my industry career, while working as a senior software engineer on a Bengali grammar and spell-checking project, I became fascinated by NLP and machine learning even though I had never taken a formal machine-learning course. I began teaching myself through online courses, technical materials, and sustained practice, spending months building the mathematical and conceptual foundations I was missing. That experience showed me what independent learning can make possible. I therefore do not view a successful computer science education as simply accumulating facts, memorizing algorithms, or learning the syntax of current technologies. Students should leave a course able to learn on their own, adapt what they know to unfamiliar situations, and approach difficult problems with curiosity rather than fear. A successful course should gradually make the instructor less necessary.

Intellectual independence is difficult to observe directly, so I use a practical test for understanding: a student who truly understands a concept should be able to explain it in their own words, create an analogy they were not given, and apply the concept to a situation the course did not cover. This definition of understanding shapes how I introduce material, assess learning, and define the role of AI in the classroom.

Teaching Approach

My education taught me the difference between learning material and learning to reason with it. I remember an Information Retrieval take-home examination that required me to derive the EM update equations for a particular n-gram language model. I understood the theory, but applying it required reconstructing the reasoning for a specific problem. I spent much of the twenty-four-hour examination moving between textbooks, research papers, and my own derivations before I could solve it. The problem showed me where my understanding stopped and gave me specific questions to answer as I returned to the theory. I want students to have that same reason to engage with new material. I therefore begin with a problem that makes a concept necessary, develop the theory needed to solve it, and return to the original problem to understand why the solution works and where it may fail. In Computer Science I laboratories at UCF, I also use questions to uncover where students' understanding stops. When students could not explain what a pointer points to, I drew variables and memory addresses, then extended the diagrams to double pointers before returning to the original question. This experience encouraged me to make the underlying structure explicit when introducing linked lists through a train analogy. Students then solved the exercises independently before I showed the solutions.

An equally important part of this approach is giving students the **freedom to fail**. My first paper went through repeated rejections over roughly two years before appearing in *Transactions on Machine Learning Research*. More important than the publication was the process I developed through those failures. I learned to define the problem and research questions more carefully before designing experiments. I want students to build that kind of process rather than simply learn to avoid failure. I would give students opportunities to revise unsuccessful solutions after examining what went wrong, so that failure becomes part of developing a process they can eventually use independently. In programming and problem-solving courses, I would further support this through scaffolding where support is deliberately and visibly removed over time. Early assignments might break a problem into smaller pieces or provide intermediate reasoning steps, debugging questions, or test cases. Later assignments would remove those supports one by one. I would tell students explicitly that this is happening, so they can see not only that they are becoming independent, but how they are changing.

Assessment and Learning Environment

Grading across five courses at Auburn taught me how easily assessment can work against its own purpose. Students often optimize for the score rather than what the assessment reveals about their understanding, and a low grade can feel like a judgment of ability rather than guidance for improvement. I want assessments to reward students who can approach unfamiliar problems and explain why a solution works, rather than those who reproduce familiar patterns. I would therefore emphasize work that requires understanding: explaining solutions in one's own words, creating new analogies, or applying methods in settings the course did not cover. My research asks a related

question about AI evaluation: what does a strong benchmark score actually demonstrate about capability? I bring the same question to assessment. Grades should provide evidence of learning, not merely reward performance.

I also want students to experience me as **approachable, rigorous, and fair**. Approachability must be built rather than announced. As a student, I often had questions I never asked because I worried they were basic. I would wait for the right pause, only to find the moment had passed. Simply telling students to ask questions does not solve this, because the barrier is not always permission. I would therefore use an anonymous question channel that I address during class, giving students a way to participate without social pressure. In office hours, I would avoid starting with solutions. I would first ask what the student understands, then work through an example with the student leading while I guide with questions. The goal is to build understanding and confidence without replacing the student's reasoning.

AI in Education

Generative AI makes intellectual independence both easier to develop and easier to lose. I experience this tension myself. In unfamiliar areas, I use AI to generate examples, reframe questions, and challenge my reasoning. At the same time, I have seen enough confident but incorrect outputs that verifying claims against primary sources has become routine. I want students to develop the same balance. AI can support learning and reduce mechanical work, but conceptual decisions, verification, and explanation must remain the student's responsibility. If a student receives a correct answer but cannot explain it, that answer does not demonstrate understanding. If they can explain it, question it, and create their own analogy for why it works, then AI has supported learning rather than replaced it.

I would therefore design assignments that require students to **interrogate AI rather than simply consume its output**. In an NLP course, for example, students might develop a mathematical optimization formulation with AI assistance, then annotate it, identify errors or unsupported assumptions, and explain why the corrected objective works. The graded component would focus on what the model got right, how the student justified that judgment, and what errors or unsupported assumptions they identified and corrected, if any. This measures the student's judgment rather than the tool's output. My broader goal is **calibrated trust**: students should neither accept AI outputs blindly nor reject them automatically. Rather, they should learn to ask what evidence justifies a claim and apply that question to AI, websites, textbooks, research papers, and instructors, myself included.

Courses, Mentoring, and Future Teaching

I am prepared to contribute to core computer science courses. As a teaching assistant at Auburn University, I supported Introduction to Algorithms, Algorithms II, Computer Organization and Assembly Language, Database I, and Web Application Development; since Fall 2026, I have been teaching laboratory sections for Computer Science I at the University of Central Florida. I am especially eager to teach **Machine Learning, Natural Language Processing**, and courses on **AI and its societal implications**. In these courses, I would begin with an application or problem worth solving and introduce methods as responses to the limitations of earlier approaches. Students would use mathematics to understand why methods work and what tradeoffs they involve.

My research offers undergraduate and graduate students entry points through constructing evaluation datasets, reproducing results, auditing benchmarks for contamination, and building evaluation environments. With undergraduate researchers, I would provide well-scoped problems and gradually reduce structure; with graduate students, I would expect greater ownership of problem formulation, experimental design, and research direction while continuing to provide guidance when needed. In advising, I aim to reproduce what my advisor gave me: patience through repeated failure, meaningful guidance, and a refusal to solve the problem on my behalf. That combination helped me become an independent researcher, and I want to offer the same experience to my students.

My motivation for teaching also comes from my educational background in Bangladesh, where access to educational resources and opportunities can depend strongly on circumstances beyond a student's control. My own education expanded the opportunities available to me, and I feel a responsibility to extend that opportunity through teaching. I want students to leave my classroom able to keep learning without me, to meet hard problems without fearing the first failure, and to use the tools available to them with curiosity, judgment, and independence.